MINTRI lab assignment 2 - **Design and implement a RAG system**

Briefing

Retrieval-Augmented Generation (RAG) is an approach that combines Information Retrieval (IR) and text generation. Instead of relying only on a language model's internal knowledge, a RAG system retrieves relevant documents from a corpus and uses them to generate grounded, context-aware answers. This links directly to IR (retrieving relevant information) and Text Mining (processing, analysing, and extracting insights from text).

What to do (in groups of 2/3 students)

- Choose a use case and a RAG platform/model
- Build a working RAG prototype
- Gather and prepare a document corpus
- Implement text mining features over the retrieved results (e.g., classification, clustering, summarisation, keyword extraction, topic analysis)
- Evaluate the system's performance

Example use cases

- Supporting students learning about Information Retrieval and Text Mining, or some other academic subject of your interest
- Exploring scientific papers or course materials
- Searching and querying documents on your laptop hard disk
- Querying a company knowledge base
- Any other relevant and interesting domain (subject to approval)

Tools and frameworks

You are free to choose your tools. Examples include:

- LLMs: OpenAI models, open-source models (e.g., LLaMA, Mistral)
- RAG frameworks: LangChain, LlamaIndex, Haystack
- Vector databases: Chroma, FAISS, Weaviate, Pinecone
- FCT's IAEDU – https://iaedu.pt/pt
- Gemini for students – https://gemini.google/students/

Goal

Build a system that not only retrieves information but understands and uses it effectively.

What we are looking for

- Well-justified design choices (not just tools)
- Clear understanding of IR + RAG principles
- Ability to analyse strengths and limitations
- A working prototype, even if simple

What you have to submit:

1. GitHub repository
2. Running prototype
3. Report
4. Presentation

Assessment criteria

Criterion	Description	Weight
Problem Definition & Use Case	Clarity, relevance, and justification of the chosen application	15%
RAG Design & Implementation	Quality of system design (retrieval, generation, pipeline) and working prototype	25%
Corpus & Data Preparation	Appropriateness, quality, and preprocessing of the dataset	15%
Text Mining Features	Use of text mining techniques (e.g., summarisation, keyword extraction, topic modelling)	15%
Evaluation & Validation	Clear evaluation methodology and analysis of system performance	15%
Presentation & Report	Clarity, structure, and ability to explain and justify design decisions	15%

Sample report ToC for your guidance

Use this as a guide to draft your report, not necessarily as the final report ToC itself.

1. Introduction & Problem Definition

- Clearly describe the use case
- Explain why this problem matters
- Define the target users and their needs
- State the research/engineering goal

Example:

"We design a RAG system to support students in Information Retrieval courses by answering questions grounded in lecture materials."

2. System Overview (High-Level Architecture)

- Diagram of the RAG pipeline
- Components:
    - corpus
    - retriever
    - generator
    - text mining layer
- Justify overall design

Show:

- why RAG is appropriate for this task
- key design decisions (e.g., hybrid retrieval)

3. Corpus & Data Preparation

- Describe the dataset:
    - size, type, source
- Preprocessing steps:
    - cleaning
    - chunking strategy
- Justify decisions

Explain:

- why chunk size was chosen
- trade-offs (context vs noise)

4. Retrieval Component

- Retrieval method:
    - lexical / dense / hybrid
- Embedding model (if used)
- Vector database

Justification:

- Why this approach fits the problem
- Expected strengths/limitations

5. Generation Component

- LLM used
- Prompt design
- How context is injected

Explain:

- how hallucination is mitigated
- how answers are grounded

6. Text Mining Features

Demonstrate added value beyond basic RAG:

Examples:

- summarisation of retrieved content
- keyword extraction
- topic modelling
- clustering results

Explain:

- why these features are useful for users

7. Evaluation & Validation

Define evaluation criteria:

- Retrieval quality (e.g., relevance of documents)
- Answer quality (correctness, faithfulness)
- User usefulness

Include:

- test queries
- example outputs
- analysis of success/failure cases
- critically analyse errors
- explain *why* the system fails in some cases

8. Results & Discussion

- What worked well
- What did not work
- Limitations of the system

Reflect on:

- retrieval errors
- generation errors
- data limitations

9. Future Improvements

- Concrete, realistic improvements:
    - better retrieval (hybrid, re-ranking)
    - improved prompts
    - larger/better corpus
    - evaluation improvements

10. Conclusion
    - Summarise key contributions
    - Reflect on lessons learned

Presentation guidelines

- Clear explanation of: problem → solution → results
- Demonstration of system
- Honest discussion of limitations
- Strong justification of decisions

Common Weaknesses (to Avoid)

- Just describing tools (no justification)
- No evaluation or only superficial testing
- Ignoring failure cases
- Overcomplicated systems with no clear benefit

Pre-Submission Checklist

1. Problem & Use Case
    - Is the use case clearly defined?
    - Did we explain who the users are and what they need?
    - Did we justify why RAG is appropriate for this problem?

2. System Design
    - Did we describe the full RAG pipeline (retrieval + generation)?
    - Are our design choices justified (not just described)?
    - Did we explain why we chose:
        - retrieval method (lexical / dense / hybrid)
        - LLM
        - tools/frameworks

3. Corpus & Data

- Is the dataset clearly described (source, size, type)?
- Did we explain preprocessing steps (cleaning, chunking)?
- Did we justify chunk size / structure?

4. Retrieval Component

- Is the retrieval method clearly implemented and explained?
- Do we show example retrieved documents?
- Did we discuss strengths and limitations?

5. Generation Component

- Did we explain how the LLM is used?
- Is the prompt design described?
- Do answers clearly use retrieved context?

6. Text Mining Features

- Did we implement at least one text mining technique?
- Did we explain why it adds value?
- Are results/examples shown?

7. Evaluation & Validation

- Did we define evaluation criteria?
- Do we include test queries and outputs?
- Did we analyse:
    - correct results
    - failure cases
- Did we reflect on limitations?

8. Results & Discussion

- Did we clearly explain what worked well?
- Did we identify what didn't work?
- Did we connect results to design decisions?

9. Improvements

- Did we suggest realistic future improvements?
- Are they based on observed limitations?

10. Report Quality
    - Is the report clear, structured, and concise?
    - Are figures/diagrams included where helpful?
    - Are all claims justified or supported by examples?

11. Presentation Readiness
    - Can we clearly explain:
        - the problem
        - the system
        - the results
    - Do we have a demo or concrete examples?
    - Are we ready to answer:
        - “Why did you choose this approach?”